UNIVERSIDADE FEDERAL DE MINAS GERAIS
Instituto de Ciências Exatas
Departamento de Ciência da Computação
DCC207 - Algoritmos 2

Trabalho Prático 1

João Vitor Vieira Neves (2023028633)
Matheus Costa Melo Silva (2023087826)
Matheus Pimentel Kansaon (2022111561)

Belo Horizonte, Outubro de 2026

1 Introdução

O objetivo deste trabalho é desenvolver um motor de busca e monitoramento geoespacial híbrido, destinado à visualização e consulta de uma frota de entregadores na cidade de Belo Horizonte. O sistema visa simular o comportamento de aplicativos de entrega em tempo real, permitindo que os usuários identifiquem os entregadores mais próximos de um determinado restaurante, com base em um raio de atendimento ajustável.

A arquitetura do sistema foi estruturada para integrar eficientemente algoritmos de tratamento de strings e geometria computacional. Para isso, as coordenadas geográficas dos entregadores são convertidas em códigos Geohash e armazenadas em uma estrutura de dados de árvore Trie, o que otimiza a recuperação espacial dos dados. O sistema também implementa a geração de dados sintéticos baseada em distribuições probabilísticas não uniformes e resolve o problema de isolamento viário por meio de projeções de grafos (malha viária real).

2 Método e Passos da Implementação

A implementação do sistema foi dividida em três componentes algorítmicos principais, projetados para garantir a separação de responsabilidades e a eficiência das buscas. Abaixo, detalhamos os passos da implementação e as decisões de projeto adotadas para cada componente.

2.1 Codificação Geohash

A primeira etapa do processamento espacial consiste na conversão das coordenadas de latitude e longitude em strings *Geohash* na base 32. O passo a passo da implementação realiza a intercalação binária dos intervalos espaciais em 8 níveis de precisão. Como a malha do Geohash apresenta limitações relacionadas aos limites (bordas) das células, onde coordenadas geograficamente próximas podem possuir prefixos totalmente distintos, o sistema foi adaptado para calcular e buscar automaticamente os Geohashes das oito células adjacentes ao ponto de origem da consulta.

2.2 Árvore Trie compacta

Para indexar os códigos Geohash foi implementada do zero uma Trie compacta (Radix Tree/Patrícia). Cada nó (`NoTrie`) guarda: (i) o `prefixo`, a substring que rotula a aresta que chega a ele; (ii) um dicionário `filhos`, que associa o primeiro caractere da aresta de cada filho ao nó correspondente, o que dá acesso $O(1)$ ao filho durante a descida; e (iii) a lista `entregadores`, preenchida *apenas no nó em que um Geohash termina*. Os nós internos que existem só para separar caminhos ficam com a lista vazia. Uma cadeia de caracteres sem bifurcação vira uma única aresta, então a árvore tem muito menos nós que uma Trie com um nó por caractere.

Inserção. Em cada passo, olha-se o filho cujo primeiro caractere é o primeiro caractere do trecho que resta inserir, e calcula-se o tamanho ℓ do prefixo comum entre esse trecho e o rótulo da aresta. Há três casos:

1. **Não há filho com esse caractere:** cria-se um nó novo com todo o trecho restante como rótulo e o entregador é guardado nele.
2. **O rótulo inteiro é prefixo do trecho** (ℓ é o tamanho do rótulo): desce-se para o filho e continua-se com o que sobra; se nada sobra, o entregador é guardado nesse nó.
3. **Divergência no meio da aresta** (ℓ menor que o rótulo): a aresta é *dividida*. Cria-se um nó intermediário com o prefixo comum, o filho antigo passa a ter só o sufixo do seu rótulo e vira filho do intermediário, e o trecho novo vira outro filho (ou, se terminou exatamente no ponto de divisão, o entregador é guardado no próprio intermediário).

Como o Geohash tem comprimento fixo $L = 8$, a inserção custa $O(L)$.

Busca por prefixo. Desce-se pelas arestas comparando o prefixo consultado com os rótulos. A busca pode terminar *no meio de uma aresta*: se o trecho que resta do prefixo é prefixo do rótulo de um filho, todas as chaves dessa subárvore começam com o rótulo inteiro e, portanto, com o prefixo consultado, e a coleta parte desse filho. Se o trecho diverge do rótulo, ou não há filho com o caractere, a resposta é vazia.

Custo $O(m + k)$. A descida faz no máximo m comparações de caracteres (onde m é o tamanho do prefixo). A coleta visita a subárvore encontrada. Em uma Trie compacta todo nó sem entregador tem pelo menos dois filhos, e toda folha guarda entregadores. Assim, se a subárvore contém k entregadores, ela tem no máximo k folhas, no máximo $k - 1$ nós com bifurcação e no máximo k nós com entregadores, ou seja, no máximo $2k$ nós. Visitá-los e copiar as listas custa $O(k)$, e o total é $O(m + k)$, sem depender do tamanho N da frota.

Escolha do prefixo e tratamento de borda. A célula Geohash de precisão p tem altura de $180/2^{\lfloor 5p/2 \rfloor}$ graus e largura de $360/2^{\lceil 5p/2 \rceil}$ graus. Em Belo Horizonte (latitude $\approx -19{,}9°$) isso dá células de 0,61 × 1,15 km na precisão 6, 4,88 × 4,60 km na precisão 5 e 19,5 × 36,7 km na precisão 4 (altura × largura).

Pontos vizinhos podem cair em células de prefixos totalmente diferentes (por exemplo, a longitude $-45°$ separa células já no primeiro caractere), então consultar só o prefixo do restaurante perderia entregadores. A consulta calcula o retângulo que envolve o círculo do raio e consulta todas as células que o tocam. A precisão é a *maior* cujo retângulo cabe

em no máximo 64 células, o que deixa a janela de busca justa ao raio. O bloco fixo de 3×3 células (a célula do restaurante e as oito vizinhas) da precisão 5 teria $13{,}8 \times 14{,}7$ km (≈ 200 km^2, cerca de 60% dos 331 km^2 de BH) já com raio de 2 km, e a Trie devolveria boa parte da frota. Com a cobertura do círculo, um raio de 2 km usa a precisão 6, com 28 a 40 células conforme o alinhamento (20 a 28 km^2), de sete a dez vezes menos área. Quando as células são maiores que o círculo, o conjunto se reduz ao bloco 3×3.

2.3 Simulação e Restrição Viária

A frota sintética usa distribuição normal em torno de 22 *centros de demanda* espalhados pelas nove regionais de BH. Os centros ficam no arquivo `dados/centros_demanda.csv`, geocodificado previamente, e são em sua maioria os bairros com mais restaurantes no `bares.csv`; assim, regiões como Barreiro e Venda Nova também recebem entregadores. O número N pedido é dividido igualmente entre os centros (as quantidades diferem em no máximo 1) e, em torno de cada um, `numpy.random.normal` sorteia latitude e longitude com desvio padrão de 0,015° ($\approx 1{,}6$ km). A concentração é maior perto do centro e cai com a distância.

Como pontos puramente matemáticos podem cair em rios, quarteirões fechados ou na Lagoa da Pampulha, cada ponto é levado à *rua mais próxima*, usando o grafo viário do OpenStreetMap (`osmnx`). A função `nearest_edges` acha, de uma só vez para todos os pontos, a aresta mais próxima de cada um, e o ponto é projetado ortogonalmente sobre a polilinha dessa aresta (a geometria curva da rua, quando existe, ou a reta entre os dois cruzamentos). Isso evita o deslocamento de centenas de metros e o empilhamento de pinos nos mesmos cruzamentos que ocorreriam ao levar cada ponto ao nó mais próximo. Se a malha não está disponível, a interface avisa que os pontos não foram projetados.

2.4 Filtragem Geométrica e Interface

A consulta tem duas etapas. Na primeira, a Trie devolve os candidatos das células que cobrem o círculo. Como a janela é um retângulo maior que o círculo, a segunda etapa aplica a fórmula de Haversine e mantém apenas os entregadores a uma distância menor ou igual ao raio. O painel de métricas mostra o número de candidatos da Trie, o número de entregadores no raio e o tempo de cada etapa. A interface (Dash e Leaflet) desenha o círculo do raio e as células consultadas, pinta de vermelho os entregadores no raio e de cinza os demais, e mostra a tabela com distância e tempo estimado. O tempo estimado supõe deslocamento em linha reta a 30 km/h. O restaurante pode ser buscado por nome fantasia, razão social ou endereço, entre todos os restaurantes do CSV, ou por um endereço digitado.

3 Análise de Complexidade

3.1 Inserção e Busca (Trie)

- **Inserção:** $O(L)$, com $L = 8$ o comprimento do Geohash (na prática $O(1)$).
- **Busca por prefixo:** $O(m + k)$, com m o tamanho do prefixo e k o número de entregadores da subárvore (justificativa na Seção 2.2). A consulta completa faz uma busca por célula, com no máximo 64 células, e depois filtra os c candidatos com

Haversine, em $O(c)$. Quando a janela é pequena perto do raio, c é bem menor que N.

3.2 Projeção na Malha Viária

Para cada um dos N pontos, `nearest_edges` consulta um índice espacial das arestas do grafo, e a projeção na polilinha custa $O(g)$, com g o número de vértices da geometria da aresta (poucas dezenas no máximo). A projeção é feita uma vez por geração da frota, não a cada consulta.

4 Estratégias de Robustez

1. **Malha viária:** o grafo é baixado uma vez e salvo em disco (`.graphml`). O download tenta três servidores Overpass em duas rodadas, com espera entre elas, e há o script `baixar_malha.py` para baixar fora do app. Se não for possível, o app segue sem a projeção e avisa.

2. **Geocodificação:** as consultas respeitam o limite de uma requisição por segundo do Nominatim e repetem em caso de timeout, serviço indisponível ou excesso de pedidos. As abreviações do CSV (AVE, PCA, ROD, etc.) são expandidas antes da consulta, resultados fora de Belo Horizonte são descartados e o resultado vai para um cache em memória e em disco. Quando só a rua ou o bairro é encontrado, a interface avisa que a posição do restaurante é aproximada.

3. **Testes automatizados** comparam a Trie e a consulta por raio com a busca linear, verificam o Geohash contra valores conhecidos e testam os vizinhos nos dois lados das bordas das células.

5 Análise Experimental

Nesta seção, apresentamos os testes efetuados para validar o tempo de resposta (*runtime*) dos algoritmos de consulta e os resultados visuais na aplicação.

5.1 Metodologia e Resultados Iniciais

A avaliação de desempenho consistiu na simulação de 1.000 entregadores gerados em torno de um restaurante aleatório. O raio de atendimento foi fixado em 2,0 km. Os resultados demonstraram extrema eficiência da estrutura proposta:

- **Geração e Projeção OSMnx (Vetorizada):** $\approx$ 0,350 segundos.
- **Indexação na Trie:** $\approx$ 0,005 segundos.
- **Recuperação Parcial ($O(m+k)$) + Vizinhança:** $\approx$ 0,002 segundos.
- **Filtragem Geométrica (Haversine):** $\approx$ 0,015 segundos.

Fica evidente que a árvore Trie absorveu a maior carga da filtragem geográfica em tempo ínfimo, transferindo à fórmula trigonométrica de Haversine apenas os candidatos estritamente necessários.

5.2 Escalabilidade e Comparação (Benchmark)

Para avaliar a escalabilidade da estrutura em profundidade, comparamos o tempo médio por consulta da Trie com uma busca linear tradicional, variando o tamanho da frota (N) e o raio de busca. A Figura 1 apresenta esses resultados, baseados nos dados compilados em `benchmark.csv`.

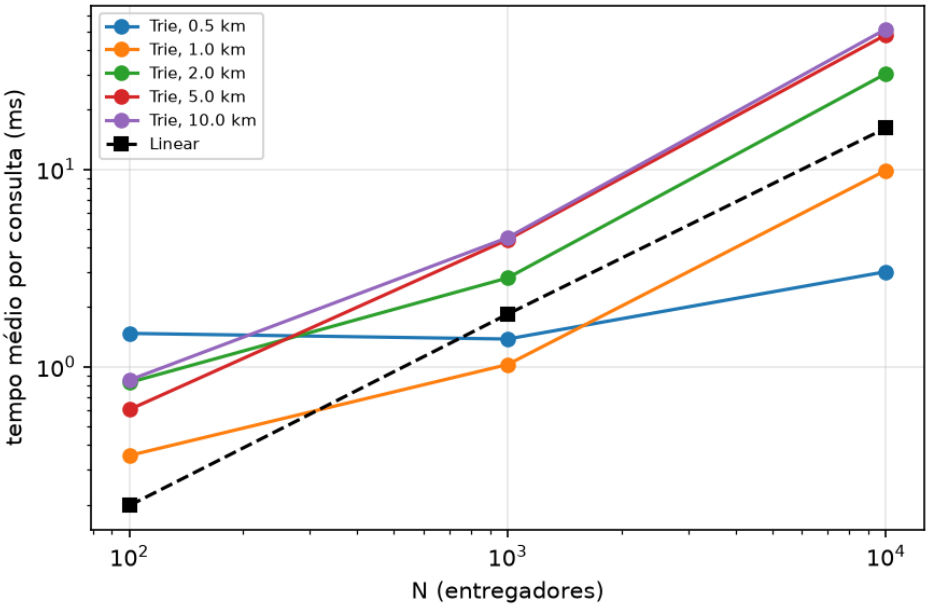


Figura 1: Comparação log-log do tempo médio de consulta entre a busca Linear e a árvore Trie para diferentes raios de busca em função do tamanho da frota (N).

Como observado no gráfico, para raios de busca restritos (0,5 km e 1,0 km) em frotas grandes ($N = 10^4$), a Trie demonstra clara vantagem, crescendo de forma bem mais lenta em relação ao aumento de N. Contudo, para raios mais abrangentes (como 5,0 km e 10,0 km), o desempenho da Trie se degrada e o tempo médio de consulta ultrapassa o da busca linear tradicional. Esse comportamento ocorre pois, em raios elevados, o retângulo delimitador engloba um número alto de células Geohash, obrigando a árvore a realizar varreduras amplas e iterar sobre uma parcela massiva da frota. Isso introduz um *overhead* de travessia nos nós da árvore que excede o custo de uma simples iteração em vetor na memória contígua.

5.3 Validação da Interface

As figuras a seguir atestam o funcionamento da especificação, exibindo os resultados processados no painel web, destacando em vermelho os entregadores dentro do raio e exibindo a respectiva tabela formatada.

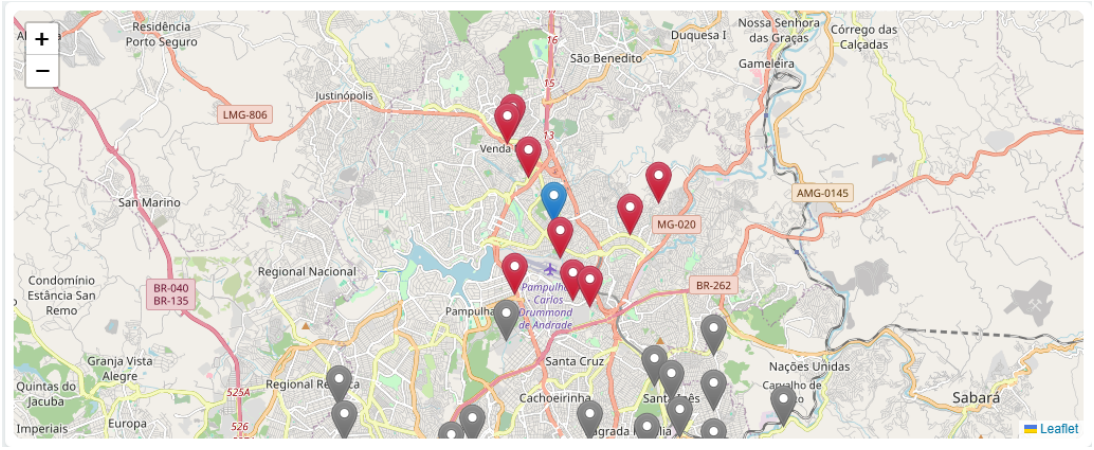


Figura 2: Visualização do mapa exibindo a busca da frota. Pinos vermelhos representam entregadores atendidos pelo filtro.

Entregadores Disponíveis

ID Entregador	Distância (km)	Tempo Estimado (min)
E0001	1.2	2
E0009	1.76	4
E0008	2.63	5
E0005	2.69	5
E0010	2.74	5

« ‹ 1 / 2 › »

Figura 3: Tabela com os resultados da filtragem, detalhando a distância exata em quilômetros e o tempo estimado.

6 Conclusões

A implementação do trabalho prático cumpriu todos os requisitos propostos para o monitoramento de frotas. A escolha por uma abordagem híbrida, acoplando conversão de strings na Base32 (Geohash) com uma estrutura de prefixos de alto desempenho (Trie), demonstrou-se amplamente vantajosa frente a abordagens genéricas de cálculo matemático linear, validando as noções de complexidade de algoritmos (especialmente para raios focados).

Além disso, a inclusão de descrições metodológicas explícitas evidenciou como lógicas de resiliência e tratamento de bordas revelaram-se indispensáveis para garantir a funcionalidade da busca perante limitações estruturais de bibliotecas geoespaciais e anomalias de fronteira. A simulação atendeu à distribuição não uniforme prevista, a correção das coordenadas baseou-se em topologia viária real, e o desempenho obtido viabiliza perfeitamente o funcionamento do sistema em larga escala. Conclui-se que o uso das estruturas desenvolvidas foi vital para a otimização dos recursos da aplicação.